DIVA

Damn Insecure and Vulnerable App — Security Assessment Writeup

Application	jakhar.aseem.diva (DIVA Beta)
Author	Aseem Jakhar
Platform	Android (Genymotion Emulator — vbox86p)
Tools Used	ADB, JADX-GUI, SQLite3, PowerShell, Logcat
Date	September 2026
Challenges	11 Vulnerabilities Documented

Table of Contents

1. Insecure Logging

2. Hardcoding Issues — Part 1

3. Insecure Data Storage — Part 1 (SharedPreferences)

4. Insecure Data Storage — Part 2 (SQLite Database)

5. Insecure Data Storage — Part 3 (Temporary File)

6. Insecure Data Storage — Part 4 (External Storage)

7. Input Validation Issues — Part 1 (SQL Injection)

8. Input Validation Issues — Part 2 (Local File Read via WebView)

9. Access Control Issues — Part 1 (Vendor API Credentials)

10. Access Control Issues — Part 2 (Tveeter API Credentials)

11. Hardcoding Issues — Part 2 (Native Library)

Challenge 1 — Insecure Logging

> **Objective:** Find out what is being logged, where, and how. Identify the vulnerable code.
> **Hint:** Insecure logging occurs when developers intentionally or unintentionally log sensitive information such as credentials, session IDs, financial details, etc.

Steps to Reproduce

1. Opened the **"1. Insecure Logging"** challenge in the DIVA app.
2. Entered a test credit card number (`77777`) in the input field and pressed **CHECK OUT**.
3. On the ADB shell, ran `logcat` with a grep filter to capture sensitive data.

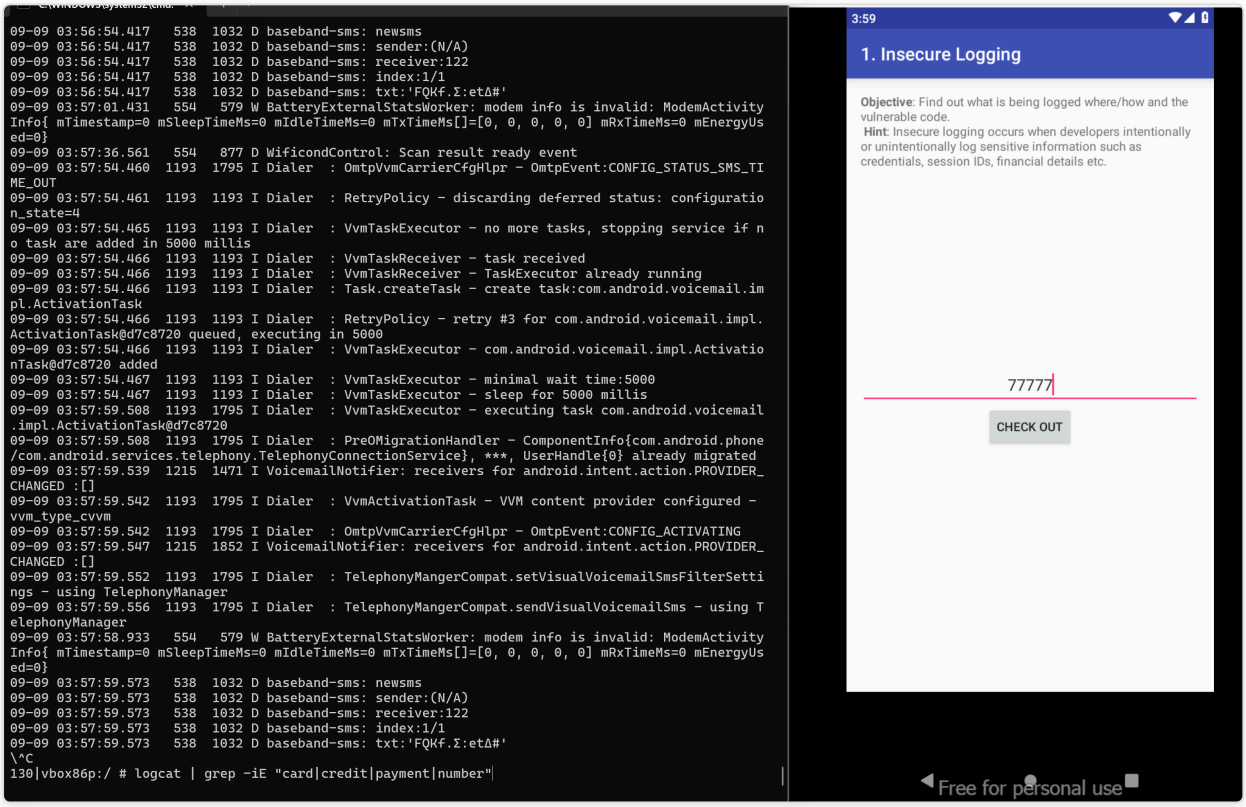


Figure 1.1 — DIVA Insecure Logging challenge with logcat grep command running

```
logcat | grep -iE "card|credit|payment|number"
```

The filtered logcat output revealed the credit card number being logged in plaintext:



Figure 1.2 — Logcat output showing plaintext credit card numbers under tag `diva-log`

```
E diva-log: Error while processing transaction with credit card: 77777
E diva-log: Error while processing transaction with credit card: 7777777
```

> ⚠ **Finding:** The app logs the **full credit card number in plaintext** to the Android system log under the tag `diva-log` at the ERROR level. Any app with `READ_LOGS` permission or any user with ADB access can read this data.

> ✓ **Remediation:**
>
> - Never log sensitive data (credentials, PII, financial info).
> - Use log-scrubbing or redaction utilities in production builds.
> - Strip verbose/debug logging before release using ProGuard rules or `BuildConfig.DEBUG` checks.

Challenge 2 — Hardcoding Issues Part 1

> **Objective:** Find out what is hardcoded and where.
> **Hint:** Developers sometimes hardcode sensitive information for ease.

Steps to Reproduce

1. Opened the APK in **JADX-GUI** and navigated to `HardcodeActivity.java`.
2. Found the hardcoded key check on line 19.

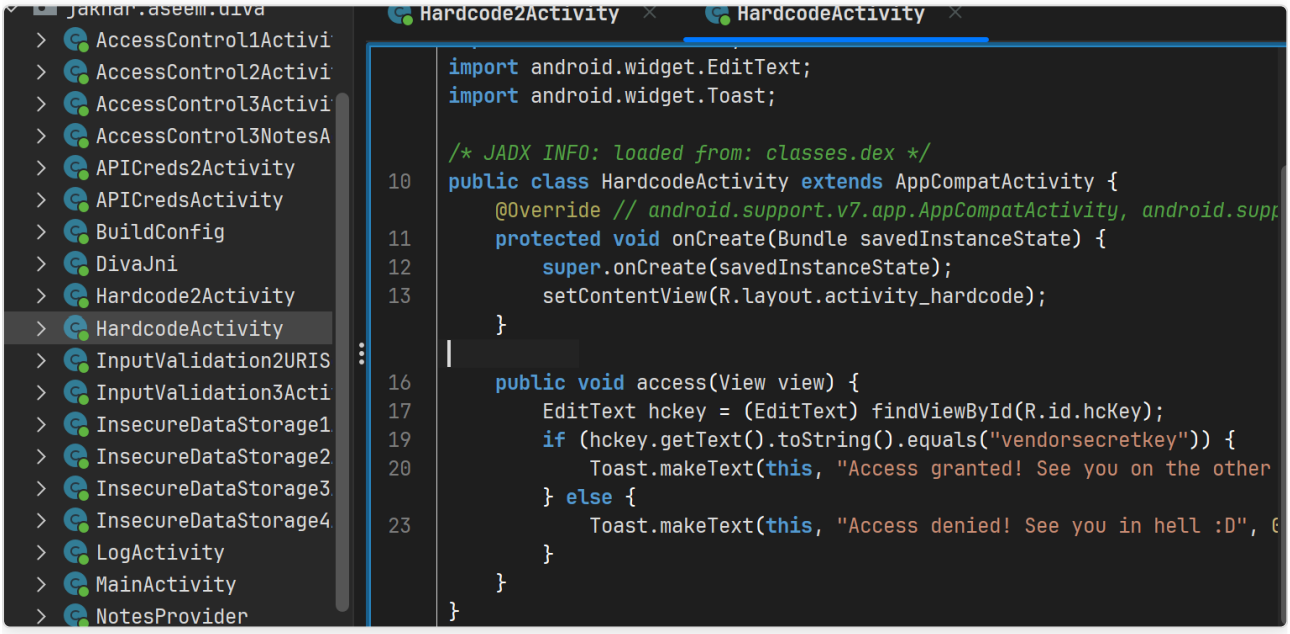


Figure 2.1 — JADX-GUI showing the hardcoded vendor secret key in `HardcodeActivity.java`

```
if (hckey.getText().toString().equals("vendorsecretkey"))
```

3. Entered `vendorsecretkey` in the app's input field and pressed **ACCESS**.
4. Received a Toast message: **"Access granted!"**

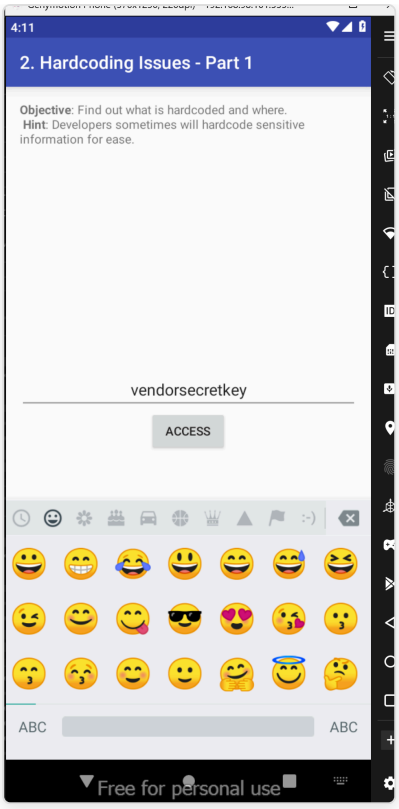


Figure 2.2 — Successfully bypassed the access check using the hardcoded key `vendorsecretkey`

> ⚠ **Finding:** The vendor secret key `vendorsecretkey` is **hardcoded in plaintext** in the Java source. Anyone who decompiles the APK can extract it immediately.

> ✓ **Remediation:**
>
> - Do not store secrets in client-side code.
> - Use server-side authentication and token-based access.
> - If local secrets are unavoidable, use the Android Keystore system.

Challenge 3 — Insecure Data Storage Part 1

> **Objective:** Find out where/how credentials are stored and the vulnerable code.
> **Hint:** Insecure data storage results from storing confidential information insecurely — poor encryption, plaintext, access control issues, etc.

Steps to Reproduce

1. Opened the **"3. Insecure Data Storage - Part 1"** challenge.
2. Entered username `okay` and password `not okay`, then pressed **SAVE**.

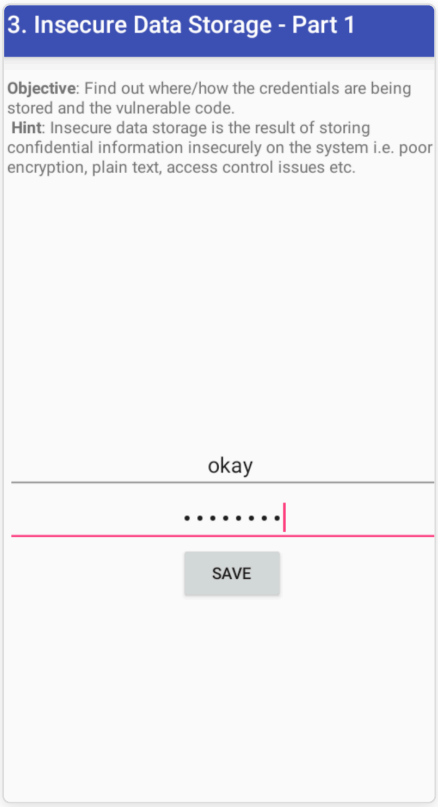


Figure 3.1 — Entering credentials in the Insecure Data Storage Part 1 challenge

3. On the ADB shell, navigated to the app's SharedPreferences directory and read the stored file:

```
127|vbox86p:/data/data/jakhar.aseem.diva # ls
cache code_cache databases lib shared_prefs uinfo16940747295304647547tmp
vbox86p:/data/data/jakhar.aseem.diva # cd shared_prefs
vbox86p:/data/data/jakhar.aseem.diva/shared_prefs # ls
jakhar.aseem.diva_preferences.xml
vbox86p:/data/data/jakhar.aseem.diva/shared_prefs # cat jakhar.aseem.diva_preferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="password">not okay</string>
    <string name="user">okay </string>
</map>
vbox86p:/data/data/jakhar.aseem.diva/shared_prefs #
```

Figure 3.2 — SharedPreferences XML file containing plaintext username and password

```
cd /data/data/jakhar.aseem.diva/shared_prefs
cat jakhar.aseem.diva_preferences.xml

<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="password">not okay</string>
    <string name="user">okay </string>
</map>
```

> ⚠ **Finding:** Credentials are stored in **plaintext** in `SharedPreferences` XML files. On a rooted device, these files are directly readable from the filesystem.

> ✓ **Remediation:**
>
> - Encrypt sensitive data before storing (e.g., using `EncryptedSharedPreferences` from Jetpack Security).
> - Use the Android Keystore for key management.
> - Avoid storing credentials locally; prefer tokens with expiry.

Challenge 4 — Insecure Data Storage Part 2

> **Objective:** Find out where/how credentials are stored and the vulnerable code.
> **Hint:** Insecure data storage — poor encryption, plaintext, access control issues.

Steps to Reproduce

1. Entered credentials and saved them in the **"4. Insecure Data Storage - Part 2"** challenge.
2. Used ADB to pull the SQLite database from the device:

```
adb -s 192.168.98.101:5555 pull /data/data/jakhar.aseem.diva/databases/ids2
```

3. Opened the database with `sqlite3` and queried the `myuser` table:

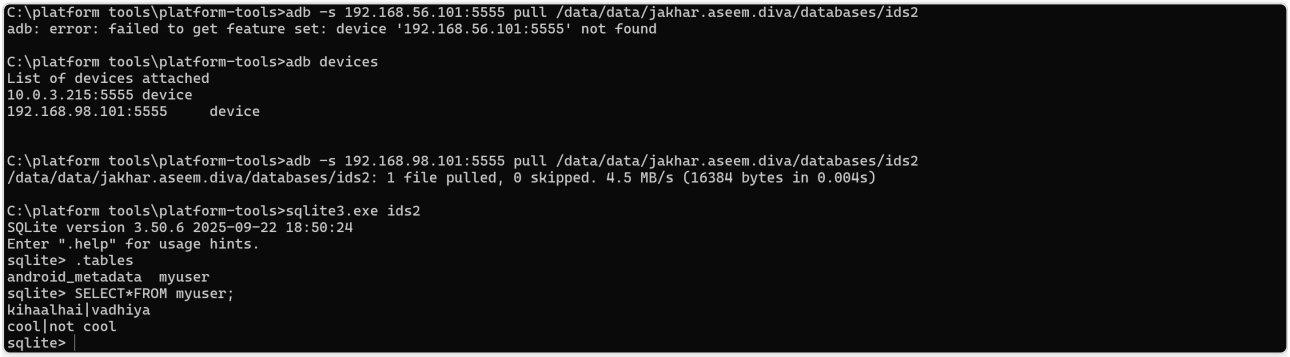


Figure 4.1 — SQLite database `ids2` containing plaintext credentials in the `myuser` table

```
sqlite3.exe ids2
.tables
→ android_metadata  myuser

SELECT * FROM myuser;
→ kihaalhai|vadhiya
→ cool|not cool
```

> ⚠ **Finding:** Credentials are stored in a **local SQLite database (`ids2`)** in the `myuser` table with **no encryption**. Username and password are plaintext columns.

> ✓ **Remediation:**
>
> - Use SQLCipher or equivalent to encrypt the database at rest.

> - Store hashed passwords if local authentication is required.

Challenge 5 — Insecure Data Storage Part 3

> **Objective:** Find out where/how credentials are stored and the vulnerable code.

Steps to Reproduce

1. Opened the **"5. Insecure Data Storage - Part 3"** challenge.
2. Entered username `hello` and password `not hello`, then pressed **SAVE**.

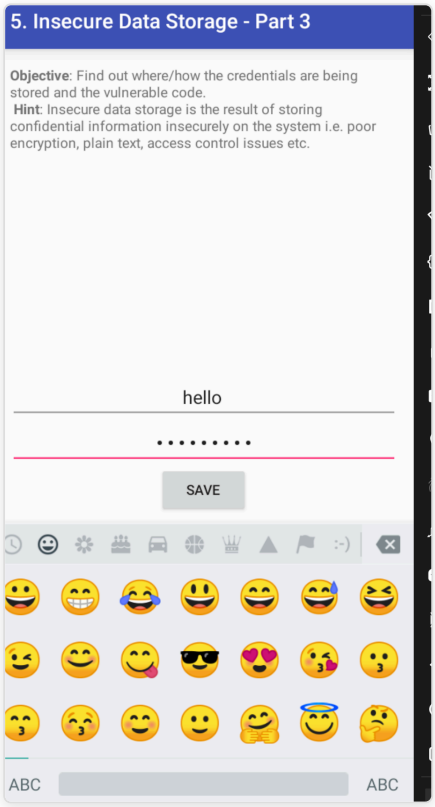


Figure 5.1 — Entering credentials in the Insecure Data Storage Part 3 challenge

3. On the ADB shell, found and read a temporary file in the app's data directory:

```
C:\platform tools\platform-tools>adb -s 192.168.98.101:5555 shell
vbox86p:/ # cd /data/data/jakhar.aseem.diva
vbox86p:/data/data/jakhar.aseem.diva # ls
cache code_cache databases lib shared_prefs uinfo1694074729530464754tmp uinfo6676645149894297904tmp
vbox86p:/data/data/jakhar.aseem.diva # cd uinfo6676645149894297904tmp
/system/bin/sh: cd: /data/data/jakhar.aseem.diva/uinfo6676645149894297904tmp: Not a directory
2|vbox86p:/data/data/jakhar.aseem.diva # cat uinfo6676645149894297904tmp
hello :not hello
vbox86p:/data/data/jakhar.aseem.diva #
```

Figure 5.2 — Temporary file `uinfo*tmp` containing plaintext credentials

```
cd /data/data/jakhar.aseem.diva
cat uinfo6676645149894297904tmp
→ hello :not hello
```

⚠ **Finding:** Credentials are stored in a **temporary file** (`uinfo*tmp`) in the app's private data directory in **plaintext**. On a rooted device, these are trivially accessible.

✓ **Remediation:**

- Do not write credentials to temporary files.
- If temp files are necessary, encrypt content and delete immediately after use.

Challenge 6 — Insecure Data Storage Part 4

> **Objective:** Find out where/how credentials are stored and the vulnerable code.

Steps to Reproduce

1. Opened the **"6. Insecure Data Storage - Part 4"** challenge.
2. Entered username `well` and password `not well`, then pressed **SAVE**.



Figure 6.1 — Entering credentials in the Insecure Data Storage Part 4 challenge

3. On the ADB shell, navigated to the SD card and found a hidden file:

```
vbox86p:/ # ls
acct       d           init                init.zygote32.rc        plat_property_contexts root     system            vendor_file_contexts
bin        data        init.environ.rc     mnt                     plat_seapp_contexts    sbin     tmp               vendor_hwservice_contexts
bugreports default.prop init.rc             odm                     plat_service_contexts  sdcard   ueventd.rc        vendor_property_contexts
cache      dev         init.usb.configfs.rc oem                    proc                   sepolicy ueventd.vbox86.rc vendor_seapp_contexts
charger    etc         init.usb.rc         plat_file_contexts      product                storage  var               vendor_service_contexts
config     fstab.vbox86 init.vbox86.rc     plat_hwservice_contexts rom.trace              sys      vendor            vndservice_contexts
vbox86p:/ # cd sdcard
vbox86p:/sdcard # ls
Alarms DCIM Download Movies Music Notifications Pictures Podcasts Ringtones
vbox86p:/sdcard # ls -ls
total 72
8 drwxrwx--x 2 root sdcard_rw 4096 2026-08-05 05:32 Alarms
8 drwxrwx--x 2 root sdcard_rw 4096 2026-08-05 05:32 DCIM
8 drwxrwx--x 2 root sdcard_rw 4096 2026-08-05 05:32 Download
8 drwxrwx--x 2 root sdcard_rw 4096 2026-08-05 05:32 Movies
8 drwxrwx--x 2 root sdcard_rw 4096 2026-08-05 05:32 Music
8 drwxrwx--x 2 root sdcard_rw 4096 2026-08-05 05:32 Notifications
8 drwxrwx--x 2 root sdcard_rw 4096 2026-08-05 05:32 Pictures
8 drwxrwx--x 2 root sdcard_rw 4096 2026-08-05 05:32 Podcasts
8 drwxrwx--x 2 root sdcard_rw 4096 2026-08-05 05:32 Ringtones
vbox86p:/sdcard # ls -la
total 92
drwxrwx--x 11 root sdcard_rw 4096 2026-09-09 03:41 .
drwx--x--x  4 root sdcard_rw 4096 2026-08-05 05:32 ..
-rw-rw----  1 root sdcard_rw   14 2026-09-09 04:34 .uinfo.txt
drwxrwx--x  2 root sdcard_rw 4096 2026-08-05 05:32 Alarms
drwxrwx--x  2 root sdcard_rw 4096 2026-08-05 05:32 DCIM
drwxrwx--x  2 root sdcard_rw 4096 2026-08-05 05:32 Download
drwxrwx--x  2 root sdcard_rw 4096 2026-08-05 05:32 Movies
drwxrwx--x  2 root sdcard_rw 4096 2026-08-05 05:32 Music
drwxrwx--x  2 root sdcard_rw 4096 2026-08-05 05:32 Notifications
drwxrwx--x  2 root sdcard_rw 4096 2026-08-05 05:32 Pictures
drwxrwx--x  2 root sdcard_rw 4096 2026-08-05 05:32 Podcasts
drwxrwx--x  2 root sdcard_rw 4096 2026-08-05 05:32 Ringtones
vbox86p:/sdcard # cat .uinfo.txt
well:not well
vbox86p:/sdcard #
```

Figure 6.2 — Hidden file `.uinfo.txt` on SD card containing plaintext credentials

```
cd /sdcard
ls -la
→ -rw-rw---- 1 root sdcard_rw 14 2026-09-09 04:34 .uinfo.txt

cat .uinfo.txt
→ well:not well
```

> ⚠ **Finding (Critical):** Credentials are stored on **external storage (SD card)** as a hidden `.uinfo.txt` file. External storage is **world-readable** — any app with `READ_EXTERNAL_STORAGE` permission can access it. This is the most severe insecure storage variant.

> ✓ **Remediation:**
>
> - Never write sensitive data to external storage.
> - Use internal storage with proper file permissions.
> - Encrypt all stored credentials using Android Keystore-backed keys.

Challenge 7 — Input Validation Issues Part 1 (SQL Injection)

> **Objective:** Access all user data without knowing any username. Three users exist by default; dump all with a single malicious search.
> **Hint:** Improper or no input validation issues arise when input is not filtered or validated before use.

Source Code Analysis

Using JADX-GUI, analyzed `SQLInjectionActivity.java`. The database is pre-populated with three users:

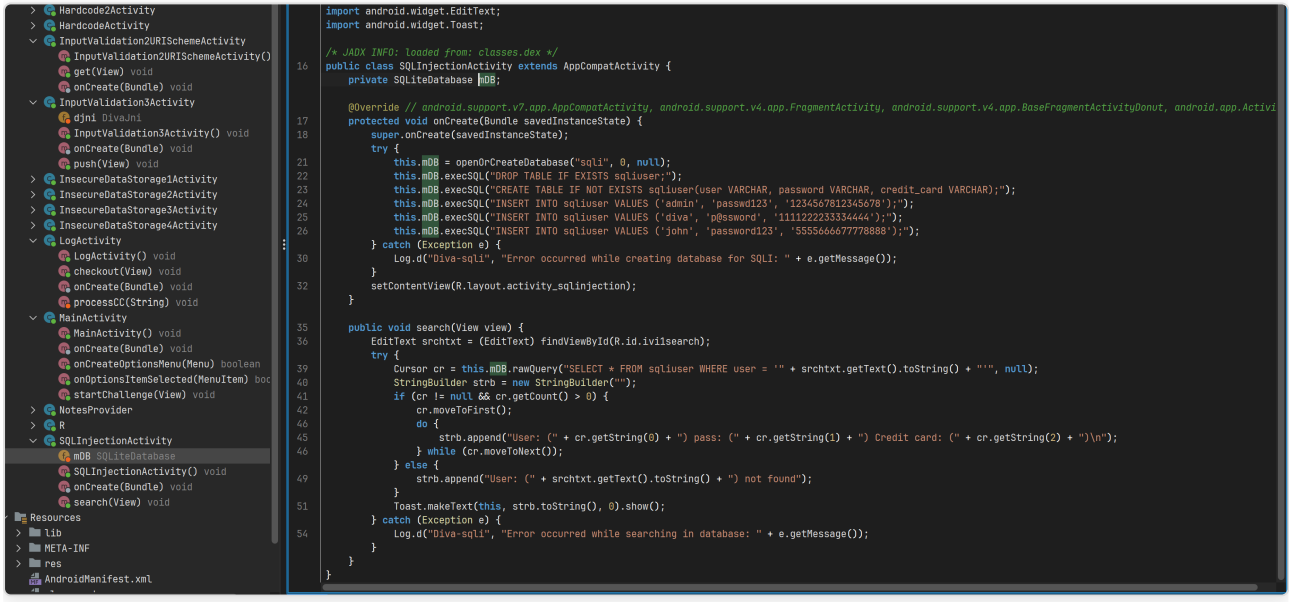


Figure 7.1 — JADX-GUI: `SQLInjectionActivity.java` showing hardcoded user data and vulnerable `rawQuery()`

```
// Database setup
this.mDB.execSQL("INSERT INTO sqliuser VALUES ('admin', 'passwd123',
'1234567812345678');");
this.mDB.execSQL("INSERT INTO sqliuser VALUES ('diva', 'p@ssword',
'11112222333344444');");
this.mDB.execSQL("INSERT INTO sqliuser VALUES ('john', 'password123',
'5555666677778888');");

// Vulnerable search query (line 39) - direct string concatenation!
```

```
Cursor cr = this.mDB.rawQuery("SELECT * FROM sqliuser WHERE user = '"
    + srchtxt.getText().toString() + "'", null);
```

Steps to Reproduce

1. First searched for `admin` legitimately to confirm functionality:

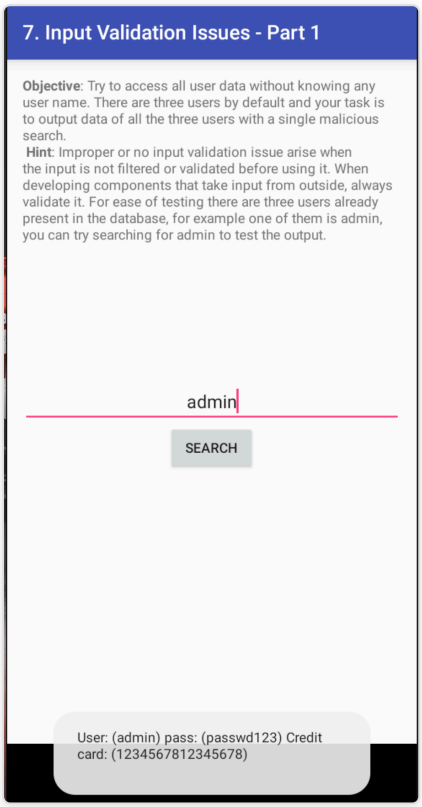


Figure 7.2 — Legitimate search for "admin" returning one record

2. Entered the SQL injection payload: `' OR '1'='1`
3. The app dumped **all three users' records** including passwords and credit card numbers:

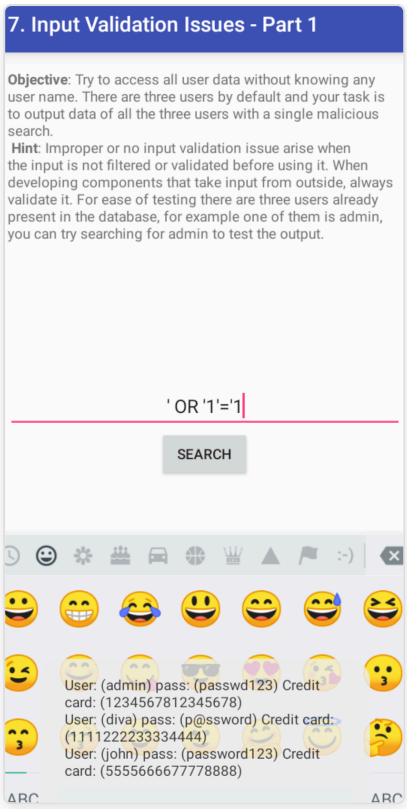


Figure 7.3 — SQL injection with `' OR '1'='1` dumping all user records

```
User: (admin) pass: (passwd123) Credit card: (1234567812345678)
User: (diva) pass: (p@ssword) Credit card: (1111222233334444)
User: (john) pass: (password123) Credit card: (5555666677778888)
```

> ⚠ **Finding (Critical):** The app uses `rawQuery()` with **string concatenation**, making it trivially vulnerable to SQL injection. A single injection payload dumps all usernames, passwords, and credit card numbers.

> ✓ **Remediation:**
>
> - Use **parameterized queries** (prepared statements):
>
> ```
> Cursor cr = this.mDB.rawQuery(
>     "SELECT * FROM sqliuser WHERE user = ?",
>     new String[]{userInput}
> );
> ```
>
> - Validate and sanitize all user input.
> - Apply the principle of least privilege to database operations.

Challenge 8 — Input Validation Issues Part 2 (Local File Read)

> **Objective:** Access sensitive information apart from a web URL.
> **Hint:** Improper or no input validation issues arise when input is not filtered or validated before use.

Steps to Reproduce

1. Opened the **"8. Input Validation Issues - Part 2"** challenge, which presents a URL input field with a **VIEW** button (WebView-based browser).
2. Instead of a web URL, entered a `file://` URI pointing to a local file:

```
file:///data/data/jakhar.aseem.diva/uinfo16940747295304647754tmp
```

3. Pressed **VIEW**. The WebView successfully loaded and displayed the local file contents:

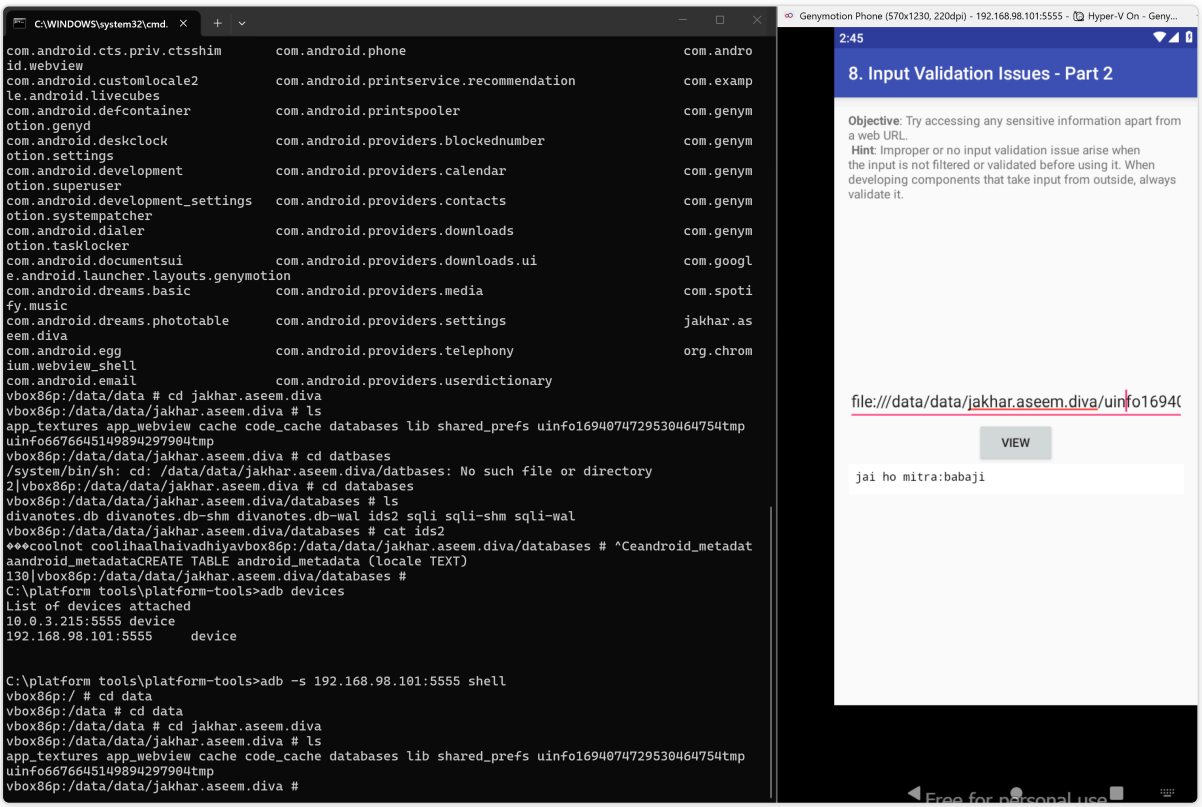


Figure 8.1 — WebView loading a local file via `file://` URI, displaying stored credentials

```
jai ho mitra:babaji
```

⚠ **Finding:** The WebView does not validate or restrict the URI scheme — it allows `file://` protocol access. This enables reading **any file accessible to the app** on the local filesystem, including credentials stored by other challenges.

✓ **Remediation:**

- Validate URL input and restrict schemes to `http://` and `https://` only.
- Disable file access in WebView:

```
webView.getSettings().setAllowFileAccess(false);
webView.getSettings().setAllowFileAccessFromFileURLs(false);
```

Challenge 9 — Access Control Issues Part 1 (Vendor API Credentials)

> **Objective:** Find hardcoded API credentials in the app.

Steps to Reproduce

1. Opened the **Vendor API Credentials** screen:

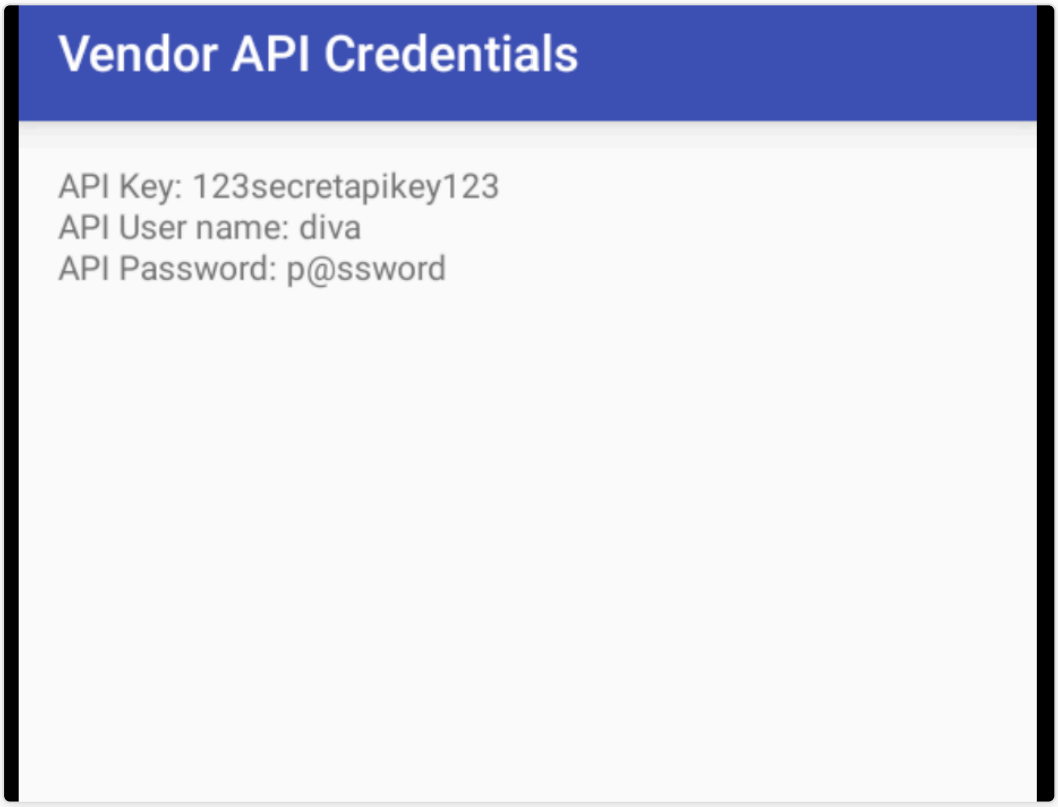


Figure 9.1 — Vendor API Credentials displayed in the app

```
API Key: 123secretapikey123
API User name: diva
API Password: p@ssword
```

2. Confirmed via JADX-GUI — `APICredsActivity.java` :

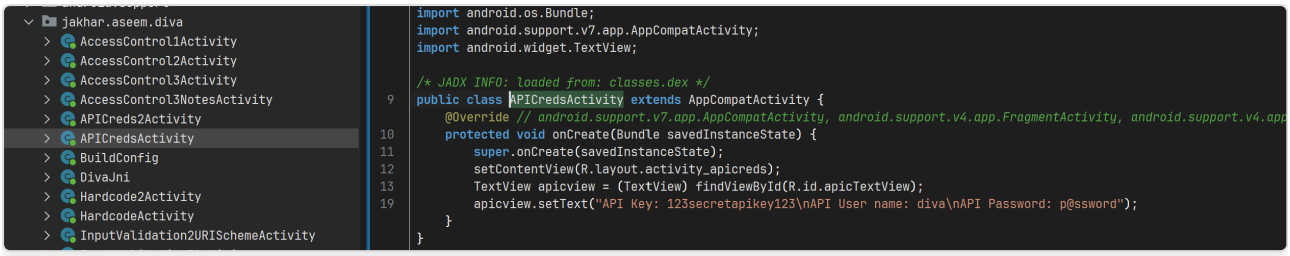


Figure 9.2 — JADX-GUI: Hardcoded API credentials in `APICredsActivity.java` (line 19)

```
apicview.setText("API Key: 123secretapikey123\nAPI User name: diva\nAPI Password: p@ssword");
```

> ⚠ **Finding:** API credentials are **hardcoded directly in the Activity source code** as a string literal. Decompiling the APK immediately reveals all API keys.

> ✓ **Remediation:**
>
> - Never embed API keys in client-side code.
> - Fetch credentials from a secure backend at runtime.
> - Use API key restrictions and rotate keys regularly.

Challenge 10 — Access Control Issues Part 2 (Tveeter API Credentials)

> **Objective:** Find hardcoded API credentials for a second service and bypass the PIN protection.

Steps to Reproduce

1. Opened the **Tveeter API Credentials** screen:

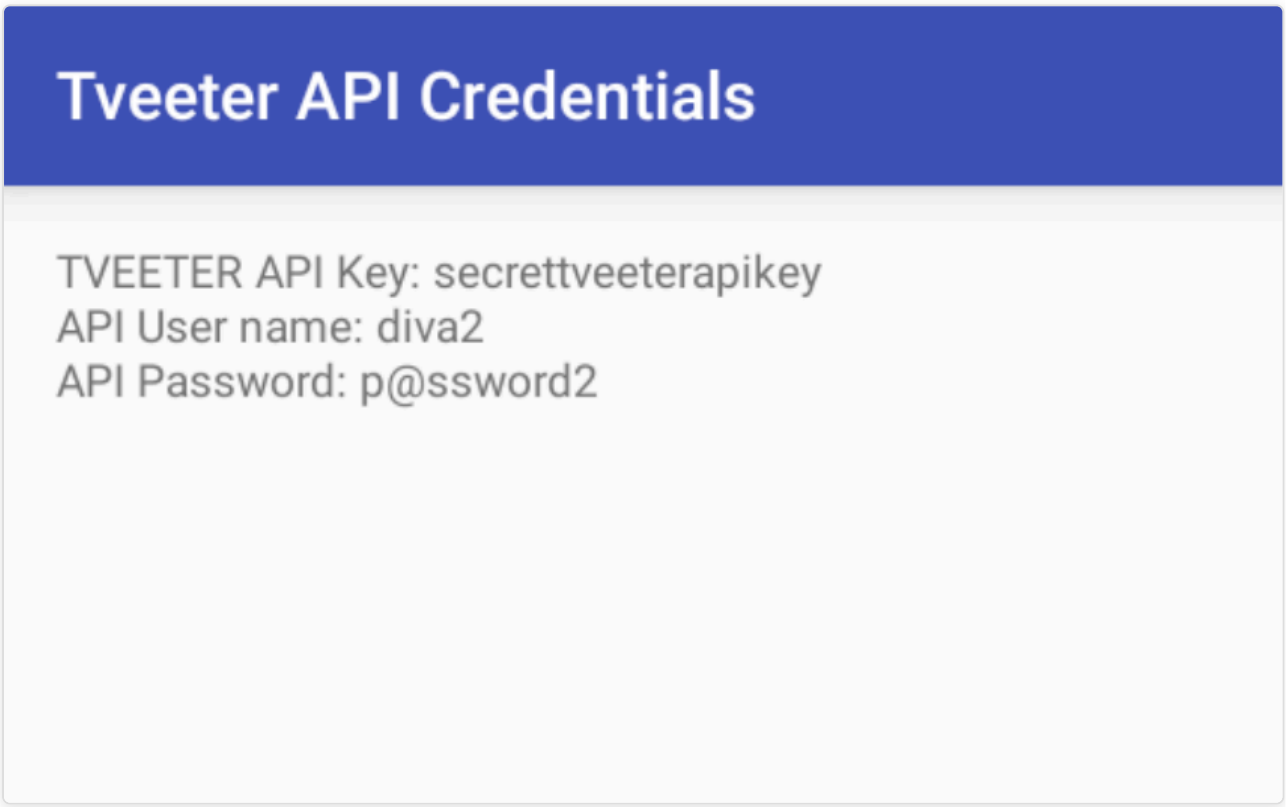


Figure 10.1 — Tveeter API Credentials displayed in the app

```
TVEETER API Key: secrettveeterapikey
API User name: diva2
API Password: p@ssword2
```

2. Analyzed the source code in JADX-GUI — `APICreds2Activity.java` :

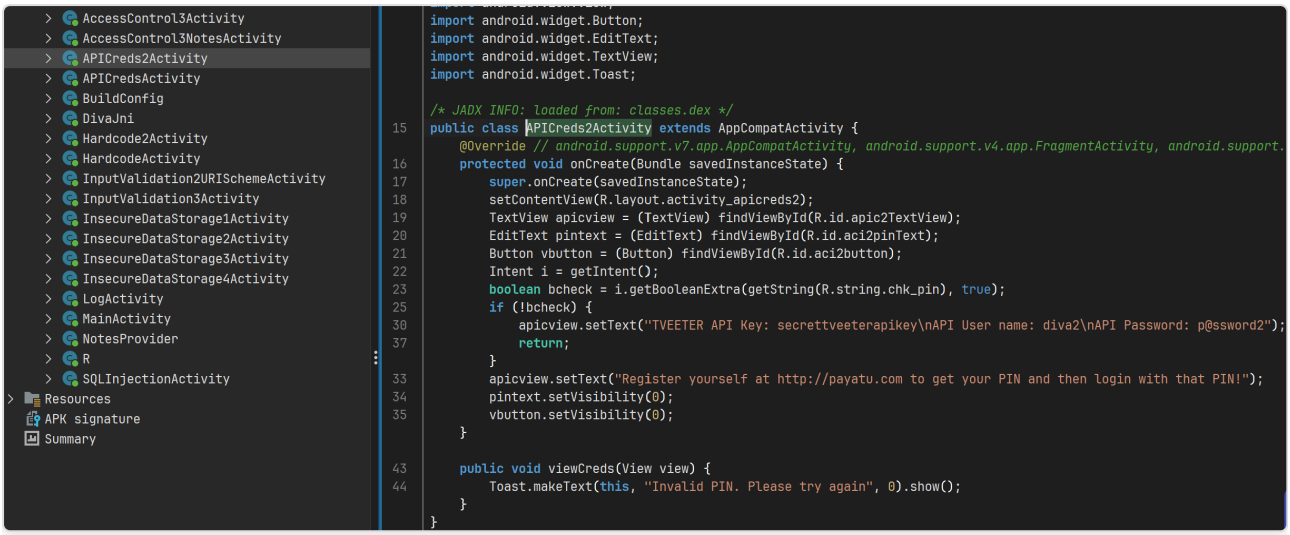


Figure 10.2 — JADX-GUI: `APICreds2Activity.java` showing PIN bypass and hardcoded credentials

```
boolean bcheck = i.getBooleanExtra(getString(R.string.chk_pin), true);
if (!bcheck) {
    apicview.setText("TVEETER API Key: secrettveeterapikey\n
        API User name: diva2\nAPI Password: p@ssword2");
    return;
}
// ...
public void viewCreds(View view) {
    Toast.makeText(this, "Invalid PIN. Please try again", 0).show();
}
```

> ⚠ **Finding:** API credentials are **hardcoded in source**. The PIN protection is entirely cosmetic — `viewCreds()` always shows "Invalid PIN", and credentials are displayed when the activity is launched with `chk_pin = false`. The access control can be bypassed by launching the activity directly with the correct intent extra.

> ✓ **Remediation:**
>
> - Implement server-side access control for sensitive resources.
> - Never rely on client-side checks for authorization.
> - Store secrets on the backend; provide them only after proper authentication.

Challenge 11 — Hardcoding Issues Part 2 (Native Library)

> **Objective:** Find the hardcoded secret in the native library.

Steps to Reproduce

1. Attempted to extract the APK on the device using `apktool` and `unzip` — both unavailable:

```
C:\platform tools\platform-tools>adb -s 192.168.98.101:5555 shell
vbox86p:/ # apktool d diva-beta.apk -o diva_extracted
/system/bin/sh: apktool: not found
127|vbox86p:/ # unzip# Unzip the APK directly into a destination folder d diva-beta.apk -o diva
/system/bin/sh: unzip#: not found
127|vbox86p:/ # unzip diva-beta.apk -d diva_extracted
unzip: can't open diva-beta.apk[.zip]
1|vbox86p:/ #
```

Figure 11.1 — Failed APK extraction attempts on the emulator

2. Extracted the APK on the host machine and located the native library at `diva_extracted\lib\x86\libdivajni.so` (5,164 bytes).
3. Attempted `strings`, `cat`, and `grep` on Windows CMD — unavailable. Switched to PowerShell:

```
C:\platform tools\platform-tools\diva_extracted\lib\x86>dir
 Volume in drive C is Windows
 Volume Serial Number is F012-CCDA

 Directory of C:\platform tools\platform-tools\diva_extracted\lib\x86

22-09-2026  13:57    <DIR>          .
22-09-2026  13:57    <DIR>          ..
31-12-2015  14:37             5,164 libdivajni.so
               1 File(s)          5,164 bytes
               2 Dir(s)  44,455,772,160 bytes free

C:\platform tools\platform-tools\diva_extracted\lib\x86>cat libdivajni.so
'cat' is not recognized as an internal or external command,
operable program or batch file.

C:\platform tools\platform-tools\diva_extracted\lib\x86>Select-String -Path "libdivajni.so" -Patt
ern "vendor", "olsd"
'Select-String' is not recognized as an internal or external command,
operable program or batch file.

C:\platform tools\platform-tools\diva_extracted\lib\x86>strings libdivajni.so
'strings' is not recognized as an internal or external command,
operable program or batch file.

C:\platform tools\platform-tools\diva_extracted\lib\x86>strings libdivajni.so | grep -E "olsd|ven
dor|key"
'strings' is not recognized as an internal or external command,
operable program or batch file.

C:\platform tools\platform-tools\diva_extracted\lib\x86>strings libdivajni.so | grep -E "olsd|ven
dor|key"
'strings' is not recognized as an internal or external command,
operable program or batch file.

C:\platform tools\platform-tools\diva_extracted\lib\x86>
```

Figure 11.2 — Multiple attempts to extract strings from `libdivajni.so` on Windows

```
C:\platform tools\platform-tools\diva_extracted\lib\x86>powershell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\platform tools\platform-tools\diva_extracted\lib\x86> Select-String -Path "libdivajni.so" -
Pattern "olsd"

libdivajni.so:2:        ◆◆
◆◆◆◆◆◆◆
       h◆◆◆◆◆◆◆◆◆◆◆◆◆◆h◆◆◆◆◆◆◆h
◆◆◆◆S◆◆◆n◆d$荃◆$◆◆◆◆◆◆d$[◆◆d$◆D$◆◆t◆Ѝd$
                                  Í◆◆◆'S◆Z◆◆◆d$荃
◆D◆D$ ◆D$◆◆◆◆◆◆◆$◆@◆◆◆◆d$[Ív◆◆'S◆ʕ◆◆◆d$◆◆+◆◆◆◆d[Ð◆t&◆$Ð◆◆&WVS◆◆
◆◆◆◆id$◆◆◆◆◆◆D$ ◆L$(◆◆L$◆D◆$◆◆◆◆
                                ◆◆◆◆◆◆d$[◆◆^_◆f◆WVS◆◆◆◆◆◆\d$ЋD$@◆|$
◆L$H◆◆L$◆D◆$◆◆◆◆<$◆D$◆◆◆◆◆|$!t ◆◆◆◆◆◆◆◆◆◆d$0[◆◆^_Ð◆t&◆D$.◆◆◆olsdfgad
;lh.dotdotzR|
          t◆◆◆DN4,◆◆◆$◆O
A◆T\◆◆◆6◆O
A◆t|◆◆◆OA◆◆|◆◆◆0◆x◆◆◆N◆A
                      ◆A◆O
uA◆
   DA◆4◆◆◆◆◆o◆A
              ◆A◆O@J


PS C:\platform tools\platform-tools\diva_extracted\lib\x86>
```

Figure 11.3 — PowerShell `Select-String` finding the string `olsd` in `libdivajni.so`

```
PS> Select-String -Path "libdivajni.so" -Pattern "olsd"
→ libdivajni.so:2:  ... olsdfgad ...
```

4. Opened `libdivajni.so` in a text editor, confirming the embedded string `olsdfgad` along with JNI symbols:



Figure 11.4 — `libdivajni.so` opened in text editor showing `olsdfgad` and JNI function names

```
Embedded key: olsdfgad
JNI symbols:
  - Java_jakhar_aseem_diva_DivaJni_access
  - Java_jakhar_aseem_diva_DivaJni_initiateLaunchSequence
```

> ⚠ **Finding:** The native library `libdivajni.so` contains the **hardcoded key** `olsdfgad` in its binary data. While native libraries are harder to reverse-engineer than Java code, tools like `strings`, Ghidra, or IDA Pro can extract embedded secrets.

> ✓ **Remediation:**
>
> - Do not embed secrets in native libraries.
> - Use runtime key derivation or server-side validation.
> - Combine obfuscation with anti-tampering and integrity checks (though no client-side protection is foolproof).

Summary of Vulnerabilities

#	Challenge	Vulnerability Type	Severity	Key Finding
1	Insecure Logging	Information Disclosure	High	Credit card numbers logged in plaintext via logcat
2	Hardcoding Issues Pt 1	Hardcoded Secrets	High	Vendor key `vendorsecretkey` in Java source
3	Insecure Data Storage Pt 1	Plaintext Storage	High	Credentials in SharedPreferences XML
4	Insecure Data Storage Pt 2	Plaintext Storage	High	Credentials in unencrypted SQLite DB
5	Insecure Data Storage Pt 3	Plaintext Storage	High	Credentials in temp file (`uinfo*tmp`)
6	Insecure Data Storage Pt 4	Plaintext Storage	Critical	Credentials on external storage (SD card)
7	Input Validation Pt 1	SQL Injection	Critical	`rawQuery()` with string concatenation
8	Input Validation Pt 2	Local File Inclusion	High	WebView allows `file://` URI
9	Access Control Pt 1	Hardcoded API Keys	High	API key/credentials in Activity source
10	Access Control Pt 2	Broken Access Control	High	PIN bypass + hardcoded Tweeter API creds
11	Hardcoding Issues Pt 2	Hardcoded Secrets (Native)	Medium	Key `olsdfgad` in `libdivajni.so`

Tools & Environment

Tool	Purpose
Genymotion	Android emulator (vbox86p, IP: 192.168.98.101:5555)
ADB	Device shell, file pull, logcat
JADX-GUI	APK decompilation and source code analysis
SQLite3	Database inspection
PowerShell	Binary string search (Select-String)

This writeup documents challenges completed in the DIVA (Damn Insecure and Vulnerable App) for educational and authorized security research purposes.